# MODPKICA Java 复刻版 开发助手 · 源码分析

版本：v1.26（2026-09-26）｜技术栈：Java 17+ / BouncyCastle 1.85 / PDFBox 3.0.4 ｜平台：Windows / Linux 离线

---

## 一、项目概述

MODPKICA 是一个离线可开发的完整 PKI（公钥基础设施）实验室项目：用 Java 复刻 Go 版 MODPKICA 全部功能，提供 X.509 证书签发、CRL 吊销列表、OCSP 在线状态协议、RFC3161 / Authenticode 时间戳、RFC 6962 CT 日志服务（SCT 服务管理）、WebAdmin 浏览器管理面板（含证书查看器、签名专区、调试专区）等全套能力。

设计目标

- 纯 javac 编译，无 Maven / Gradle / 网络依赖，源码改完即可 build.cmd 一键编译
- 所有加密算法走 BouncyCastle 1.85（BC185 全家桶），PDF 签名 / 验证走 PDFBox（lib/pdfbox-app-3.0.4.jar）
- 证书全流程支持 RSA / ECC（P-256/384/521）/ SM2 / PQC（ML-DSA-44/65/87） 四类算法族，哈希 SHA1 / SHA224 / SHA256 / SHA384 / SHA512 / SM3 / NULL（PQC 纯签名）
- 全部运行时数据落在 MODPKICA_BASE/PKI/ 目录，文本登记表（CERTS.txt / myCERTS.txt / CONFIG.txt / logs.log）可人工审计

主程序（7 个 + 1 个独立面板）

| 类 | 角色 | 一句话职责 |
| --- | --- | --- |
| MAIN | 网络服务主程序 | HTTP 服务：OCSP / CRL / CRT 下载 / RFC3161 / Authenticode 时间戳 / CT 日志 / 静态网页，内嵌 WebAdmin |
| Admin | 管理控制台（命令行） | init / makeroot / list / signcert / revoke / signCRL / initOCSP / initTIMESTAMP / verifyCERT |
| MakeRoot | 签发根证书 | 生成根证书 + 私钥，登记 CERTS.txt（类型 R），同步 WebPublic/CRT |
| MakeCert | 签发下级证书 | 签发中间 CA（C）/ 终端证书（E），支持 SCT 预证书流程 |
| RootGetCrl | 签发吊销列表 | 遍历全部 R / C 证书逐个生成 CRL |
| CertVerify | 证书验证 | 全字段解析、签名验证、证书链、CRL/OCSP 吊销状态、SCT 逐条验签 |
| Auto | 自动化命令 | init / revoke / restore / newcert 手工登记 |
| WebAdmin | 浏览器管理面板 | 内嵌或独立启动，15+ 页面、40 个 API 路由 |

---

## 二、目录结构

```
modpkicaforjava/
 ├─── BC185/                          # BouncyCastle 1.85 离线 jar 全家桶（bcprov/bcpkix/bcutil/bcmail）
 ├─── lib/                            # pdfbox-app-3.0.4.jar（PDF 签名 / 验证 / 时间戳嵌入）
 ├─── src/modpkica/                   # 全部源码（21 个类，约 1.2MB）
 ├─── out/                            # javac 编译输出
 ├─── build.cmd                       # Windows 一键编译脚本（支持 build.cmd [1-8] 自动运行）
 ├─── build.sh                        # Linux 一键编译脚本（支持 ./build.sh [1-8]）
 ├─── README.md                       # 项目介绍 + 全部界面截图 + 更新记录
 ├─── log_list.json                   # Google 权威 CT 日志公钥数据集 v3（离线快照）
 ├─── a.png ~ k.png / certview.png / stamp.png   # Web 面板截图
 ├─── PKI/                            # 运行时数据目录（MODPKICA_BASE 指向项目根时）
 |    ├─── CERTS.txt                   # 证书登记表（核心数据库）
 |    ├─── CONFIG.txt                  # 系统配置（WEBPORT/OCSP/CRL/CRT/CPS/SUPORT）
 |    ├─── logs.log                    # 操作日志（追加式）
 |    ├─── ROOT/                       # 根证书 序列号.crt / 序列号.key（类型 R）
 |    ├─── CA/                         # 中间证书 序列号.crt / 序列号.key（类型 C）
 |    ├─── CERT/                       # 终端证书 PEM 序列号.crt（类型 E）
 |    ├─── KEY/                        # 终端证书私钥 PEM 序列号.key（类型 E）
 |    ├─── OCSP/                       # OCSP 签名证书 序列号_ocsp.crt / 序列号_ocsp.key
 |    ├─── TIMSTAMP/                   # TSA 时间戳证书（类型_哈希.crt，如 rsa_sha256.crt）
 |    ├─── MY/                         # 我的证书（导入库）序列号.crt / 序列号.key + myCERTS.txt
 |    ├─── CTLOG/                      # CT 日志：modpkica_ct_list.json（密钥库）+ ctlogs.txt（请求日志）
 |    └─── WebPublic/                  # 对外公开目录（/CRT、/CRL、网页）
 |         ├─── CRT/                   # 全部证书副本 序列号.crt
 |         └─── CRL/                   # 吊销列表 序列号.crl / 序列号.der.crl
```

---

# 三、构建与运行

## 3.1 一键编译（离线）

Windows：build.cmd  Linux/macOS：./build.sh

脚本特性：

- 校验 JDK 17+（javac 在 PATH）；离线 classpath = BC185* + lib*
- 清理并重建 out/，javac -encoding UTF-8 全量编译 src/modpkica/*.java
- 编译成功后逐一校验 8 个主类 class 存在（MAIN/Admin/MakeRoot/MakeCert/RootGetCrl/CertVerify/Auto/WebAdmin）
- 编译失败则中止，绝不执行后续命令
- 注释全英文（避免 cmd 代码页 GBK 中文乱码导致执行错误）

编号自动运行（编译成功后自动执行对应命令，失败则不执行）：

```
build.cmd [1-8]   /   ./build.sh [1-8]
  [1] MAIN         （网络服务 + 内嵌 WebAdmin，http://localhost:8080）
  [2] WebAdmin     （仅管理面板）
  [3] Admin        （命令行管理控制台）
  [4] MakeRoot "CN=Root,O=Org,C=CN" 2048 sha256 RSA
  [5] MakeCert "CN=www.test.com" 2048 sha256 1 1 0 1 "www.test.com" null null root RSA
  [6] RootGetCrl
  [7] CertVerify cert.crt
  [8] Auto revoke 1234567890 1
```

### 3.2 环境变量与配置

- MODPKICA_BASE：PKI 根目录（AppPaths.baseDir() 解析：系统属性 modpkica.base / 环境变量 MODPKICA_BASE / 当前目录，三级回退）
- -Dfile.encoding=UTF-8：所有入口统一 UTF-8，避免中文乱码
- 管理面板端口：系统属性 modpkica.admin.port -> CONFIG.txt 的 SUPORT（默认 8080）
- MAIN 服务端口：CONFIG.txt 的 WEBPORT（默认 80）

---

## 四、模块详解（21 个类）

### 4.1 基础设施类

AppPaths（入口定位）

- baseDir()：解析 PKI 根目录（属性 -> 环境变量 -> 当前目录）

AppEnv

- getExeNames()：返回 7 个主程序的可执行名清单（对应用途提示）

Bc

- init()：注册 BouncyCastleProvider（全局唯一初始化，所有加密入口调用）

Config（CONFIG.txt 读写）

- 构造 Config(baseDir) -> load()：解析 KEY=VALUE 文本（支持 \n 换行转义、{CID} 占位）
- getCpsWithCid(certId)：CPS 替换 {CID} 占位符
- getCrlUrl() / getCrtUrl() / getOcspUrl() / getCps() / getCpsNonice() / getWebPort()：各配置项读取
- getRaw()：原始键值 Map（Web 系统配置页读写用）
- 键：WEBPORT、OCSP、CRL、CRT、CPS、CPS_Nonice、SUPORT 等

MLog（操作日志，追加式）

- append(baseDir, operation, result) / append(baseDir, operation, command, result)：写一条日志
- appendErr(baseDir, operation, command, error)：写错误日志
- 格式：时间 | 时间戳 | 操作 | 命令/提交数据 | 结果，两次写之间用两个换行分隔，只追加不删除；write() 静态 synchronized 线程安全

CertsDB（CERTS.txt 内存数据库）

- load(baseDir) / reload()：加载 / 重载登记表
- findBySerial(serial) / findByIssuerAndSerial(issuerId, serial)：序列号查询（支持不带 0 前缀容错）
- findIssuerIdBySerial(serial)：由终端证书反查颁发者 ID
- addEntry(e)：新增登记（签发证书 / 手工登记）
- revokeCert(serial, reason, time)：吊销（状态 R + 原因）
- getAllRootAndCA()：全部 R / C 颁发者（CRL、initOCSP、initTIMESTAMP 遍历）
- getAllRevoked()：全部已吊销（CRL 生成）
- getAll()：全部条目（Web 列表）
- CertEntry：序列号 / 类型 R-C-E / 状态 V-R-N / 原因 / 时间 / 颁发者 ID / 算法

PemIO（PEM 读写 + 密钥类型探测）

- readCert(path)：PEM/DER -> X509Certificate
- readPrivateKey(path)：PEM -> BC AsymmetricKeyParameter（RSA/EC/SM2/PQC 通用）
- readPrivateKeyAsSigner(path)：-> ContentSigner（签发用）
- writeCert(path, cert) / writePrivateKey(path, key, keyType)：写 PEM（PKCS#8 / EC 等按类型）
- detectKeyType(pemPath)：探测私钥算法 -> RSA / ECC / SM2 / PQC
- isPqcOid(oid) / isPqcAlgName(alg)：PQC（ML-DSA 2.16.840.1.101.3.4.3.x）判定

Sm2Util（SM2 底层）

- getSm2UidBytes()：默认用户标识 1234567812345678

- sm3(data)：SM3 摘要
- computeZa(pub, uid)：SM2 ZA 值（GB/T 32918.2）
- sm2Sign(priv, tbs)：SM3(ZA||M) 签名
- sm2Verify(pub, tbs, sig)：验签
- generateSm2JcaKeyPair()：JCA SM2 密钥对
- toEcPublicParams / toEcPrivateParams：JCA <-> BC EC 参数互转

CertUtil（证书构建核心）

- 密钥生成：generateRsaKeyPair(bits) / generateEccKeyPair(bits)（P-256/384/521）/ generateSm2KeyPair() / generatePqcKeyPair(spec)（ML-DSA-44/65/87，BC 1.85 MLDSAParameterSpec.fromName）
- normalizePqcSpec(spec)：规格名规范化（大小写/缩写容错）
- getSignatureAlgOid(keyType, hash)：按密钥类型+哈希选签名算法 OID（RSA/ECC/SM2 全套）
- readJcaPrivateKey(path)：PEM -> JCA PrivateKey
- createContentSigner(key, keyType, hash)：统一内容签名器；内部 SM2ContentSigner（SM3(ZA||M) 标准实现）
- getSubjectKeyIdentifier(pubKey)：SKI（SHA-1 截断法）
- buildSelfSignedRoot(subject, kp, ...)：自签根证书（TBS 构建 + AKI/SKI/BasicConstraints/KeyUsage/证书策略/SAN 等）
- buildSignedCert(issuerCert, issuerKey, ...)：下级证书签发（含 CA/EE 判定、EKU、策略、SCT 扩展位）

CrlUtil（CRL 生成）

- generateCrl(caCert, caKey, revokedList, hash)：X.509 v2 CRL（吊销序列号 + 原因 + 时间 + 下一个更新时间）
- generateCrlObject(...)：返回 X509CRL 对象（验证用）
- derToPem(der)：DER -> PEM（.der.crl 与 .crl 双格式）
- 内部 RevokedCert(serial, revocationTime, reason) 数据类

TimestampUtil（时间戳）

- parseRfc3161Request(der)：解析 TimeStampReq（版本 / messageImprint / nonce / certReq）
- createRfc3161Response(requestDer, ...)：完整 TimeStampResp（tstInfo + 签名 + TSA 证书），SM2 时摘要切 SM3
- createRfc3161ResponseStandard(...)：标准 BC 高层实现
- 内部 SM3DigestCalculator：SM3 DigestCalculator 适配
- createAuthenticodeTimestamp(content, ...)：微软 Authenticode 时间戳（PKCS#7 分离签名，eContent + signedAttrs 含 id-data）

OcspUtil（OCSP）

- parseRequest(der)：解析 OCSPReq（CertID：哈希算法 OID / 颁发者名哈希 / 颁发者公钥哈希 / 序列号；Nonce）
- createResponse(keyType, ...)：构造 OCSPResp（good / revoked / unknown），SM2 分支手动 ASN.1 编码（revoked [1] IMPLICIT RevokedInfo，RFC 6960）；摘要按请求哈希算法自适应（SHA1/256/384/512，SM2 固定 SM3，无摘要默认 SHA1）
- matchesIssuerHash(issuerCert, ...)：颁发者哈希匹配校验（不匹配返回无签名响应）
- unauthorizedResponse()：未知序列号无签名 unauthorized 响应

## 4.2 主程序类

MAIN（网络服务主程序）

- main()：读 CONFIG -> 注册 BC -> 启动 CRL 自动刷新 -> 启动 HTTP 服务（com.sun.net.httpserver）
- 路由：
- - POST /OCSP、/ocsp：OCSP 处理（查 CERTS.txt -> 加载 序列号_ocsp.crt/.key -> 按请求摘要算法签名响应；未知序列号 -> 无签名响应）
- - GET /CRL/{序列号}.crl、.der.crl：CRL 下载
- - GET /CRT/{序列号}.crt：证书下载
- - POST /rfc3161：RFC3161 时间戳（文档签名，SHA1/SHA256/SM3 自适应）
- - POST /timestamp（SHA256）、POST /timestamp/sha1：Authenticode 时间戳
- - 自适应根 TSA：SHA1 -> 随机 rsa_sha1 / ecc_sha1；SHA256 -> 随机 rsa_sha256 / ecc_sha256；

SM3/SM2 -> sm2_sm3

- - POST /ct/add：CT 日志服务（RFC 6962 §3.1，PEM 预证书 + 颁发者证书 -> 类型匹配选组（SM2 只选 SM2 密钥，RSA/ECDSA 选 RSA+ECDSA），随机 <=3 组 usable 密钥签名 -> 返回 SCTList）
- - GET /：静态网页（PKI/WebPublic）
- 启动横幅：打印全部服务地址 + 作者信息 + 免责声明

Admin（命令行管理控制台）

- 子命令：init、makeroot、list、signcert、revoke、signCRL、initOCSP、initTIMESTAMP、verifyCERT
- 每个 do* 方法都是 WebAdmin API 与命令行的共用入口（captureStdout 捕获输出）
- doInit(baseDir, dn, bit, hash, algo)：初始化环境（建目录 + 可选首根证书）
- doMakeRoot(...)：签发根证书（含 EV organizationIdentifier OID 版本参数 new/old）
- doSignCert(baseDir, dn, bit, hash, keyUsage, exUsage, isCa, days, dns, ips, issuer, algo)：签发证书（12 参数，对齐 Go 版）
- doRevoke / doRestore：吊销 / 恢复（R->V，原因 0，同步生成 CRL）
- doSignCrl(baseDir)：刷新全部 CRL
- doInitOcsp(baseDir, algo)：按 CERTS.txt 中 R/C 逐个签发 OCSP 签名证书（序列号_ocsp.crt/.key，从 ROOT/CA 目录读密钥）
- doInitTimestamp(baseDir, hash)：按全部根证书生成 TSA 时间戳证书（类型_哈希.crt，如 rsa_sha256.crt、sm2_sm3.crt）
- doVerifyCert(baseDir, target)：证书验证（复用 CertVerify）

MakeRoot

- run(baseDir, args)：args = DN 位数 哈希 算法 [EVOID版本]
- 流程：解析 DN（按 X.500 规范排序）-> 生成密钥对（RSA/ECC/SM2/PQC）-> 构建自签根证书（AKI=SKI、BasicConstraints CA、KeyUsage、证书策略 CPS+UserNotice、EV organizationIdentifier）-> 写 ROOT/序列号.crt + .key -> 同步 WebPublic/CRT/序列号.crt -> 登记 CERTS.txt（R）
- PQC 分支：CertUtil.generatePqcKeyPair(spec)，位长用 normalizePqcSpec 显示

MakeCert

- run(baseDir, a)：普通模式 12 参数 = DN 位数 哈希 keyUsage exUsage isCa 年限/日期 dns ips Ker

nel 颁发者ID 算法

- 流程：解析 DN -> 定位颁发者（R->ROOT 目录、C->CA 目录读 序列号.crt/.key）-> 生成主体密钥 -> 构建 TBS（AKI/SKI/BasicConstraints/KeyUsage/EKU/证书策略/SAN/AIA/CRLDP）-> 按颁发者算法 + 请求哈希签算法名（PQC 忽略哈希）
- SCT 预证书流程：签发非 CA 证书且 EKU 含服务器/客户端认证（1.3.6.1.5.5.7.3.1/2）时——先签发含 poison 扩展的预证书 -> POST 本机 /ct/add -> 取回 SCTList -> 嵌入 SCT 扩展（1.3.6.1.4.1.11129.2.4.2）二次签名生成最终证书
- 有效期：支持 年数 或 起始日期-截至日期（精确到秒）
- 输出：CERT/KEY（E）、CA（C）、ROOT 不签；同步 WebPublic/CRT；登记 CERTS.txt（E/C）

RootGetCrl

- run(baseDir)：遍历全部 R/C 证书，逐个生成独立 CRL（WebPublic/CRL/序列号.crl + .der.crl），哈希算法沿用颁发者证书（RSA/sha256、RSA/sha512、SM2/sm3 等）

CertVerify（74KB，最重验证引擎）

- run(filename)：全字段打印 + 验证
- 验证链：签名验证（含根自签）-> 证书链（终端->CA->Root，SPKI 匹配）-> 信任锚（PKI/ROOT 实验信任锚 + JAVA cacerts）-> CRL 状态（WebPublic/CRL）-> OCSP 状态（CONFIG OCSP URL）-> 各类哈希（密钥 Id SHA1/RFC-SHA1/Pin-SHA256、证书 SHA1/SHA256、签名哈希）
- SCT 逐条验签（RFC 6962）：日志公钥源 = 本地 PKI/CTLOG/modpkica_ct_list.json + 权威 log_list.json；构造 precert_entry / x509_entry 待签名数据；ECDSA（DER + 裸 r‖s）/ RSA / SM2（枚举 7,4 与 7,7）验签
- OID 中文映射：EKU、证书策略（EV/DV/OV/IV、代码签名普通/EV、TSA、文档签名标准/微软）等
- inJavaCacerts：cacerts 信任库判定

Auto

- runInit(baseDir) / revoke(baseDir, serial, reason) / restore(baseDir, serial)：自动化命令
- 支持 newcert 序列号 类型 状态 原因 时间 颁发者ID 算法 手工登记

## 4.3 专用功能类

CtLog（35KB，RFC 6962 CT 日志服务）

- 路径：keysJsonPath（PKI/CTLOG/modpkica_ct_list.json）/ logsTxtPath（ctlogs.txt）/ publicListPath（log_list.json）
-

loadPublicList / publicListTimestamp / fetchPublicList：权威日志公钥数据集读写（v3，含数据日期，可在线拉取）

- loadKeys / saveKeys：CT 日志密钥库 CRUD（v3 风格 operators[].logs[]，含 private_key、type ECDSA/RSA/SM2）
- generateKey(desc, type)：生成 ECDSA-P256 / RSA-2048 / SM2 密钥对并写入密钥库
- deleteKey / setState：删除 / 启用（usable）/ 暂停（disable）
- pickLogKeys(baseDir, sm2Precert)：类型匹配选组——SM2 预证书只选 SM2 密钥，RSA/ECDSA 预证书只选 RSA+ECDSA，随机 <=3 组 usable
- isSm2Precert(cert)：解析 SPKI named curve OID（1.2.156.10197.1.301）判定 SM2（ECC 但 SM2 曲线）
- signSctList(tbsNoPoison, issuerKeyHash, ...)：RFC 6962 §3.1 SCT 待签名数据（version + sigType + timestamp + entryType + issuer_key_hash + tbs + extensions）+ 签名 + SCTList TLS 编码；SCT 枚举 SM2=(7,4)、ECDSA=(4,3)、RSA=(4,1)
- submitPrecert：MakeCert 侧提交预证书到自身 /ct/add 并取回 SCTList
- appendRequestLog / readLogs：ctlogs.txt 追加（SCT待签名摘要||时间戳||终端SKI||终端AKI||日志公钥ID）/ 读取
- parsePemCerts / pemCert / tbsWithoutPoison / extHex：PEM 解析、去 poison TBS 重建、扩展提取

SignTool（147KB，签名专区核心）

- 证书能力：isCodeSignCert（微软代码签名 EKU 1.3.6.1.4.1.311.10.3.3 或 EV 2.23.140.1.3）/ isDocSignCert（文档签名 1.3.6.1.4.1.311.10.3.12 或通用 1.3.6.1.5.5.7.3.36）/ ekusOf / keyBits
- 加载：loadCertBySerial / loadKeyBySerial（R/C/E + MY 我的证书）/ loadChain（证书链）
- 算法映射：digestName / digestJca / digestOid / sigAlgOid / jcaSigAlg（RSA->SHA256、ECC->SHA256、SM2->SM3、PQC->NULL）
- 微软代码签名：signExe(file, keyType, cert, key, chain, tsType, tsUrl, ...)——PE/COFF 规范解析（Certificate Table、SPC Indirect Data、PKCS#7 签名 + RFC3161 或 Authenticode 时间戳嵌入）；verifyExe(exe)：完整验证（签名 + 时间戳 + 链）
- PDF 文档签名：signPdf(pdf, keyType, cert, key, chain, sealType, tsUrl, image, ...)——四种类型：
  - Adobe 标准（adbe.pkcs7.detached）：CAdES-T 签名时间戳（unsignedAttrs id-aa-timeStampToke

n，messageImprint=SHA-256(签名值)）+ DSS/LTV 同保存（签名后零增量）；纯文档时间戳（无证书）生成 DocTimeStamp（ETSI.RFC3161）+ DSS/LTV

- - 国密 GB/T 35275-2017（GM.sm2cms.detached）：SM2 证书 + 签章图，CAdES-T（SM3）+ DSS/LTV
- - 国密 GB/T 38540-2020（GM.sm2seal SES_Signature）：SESealInfo（印章）+ 时间戳字段 timeStamp（GB/T 20520）+ DSS/LTV
- - 国密 GM/T 0031-2025（GM.sm2seal）：数据结构与 38540 一致（version=4，SESv4）
- - signPdfTimestampOnly(pdf, tsUrl)：不签名仅文档时间戳
- applyLtvPdf(signedPdf, chain, baseDir)：DSS/LTV（证书链 + OCSP 响应 + CRL）叠加
- verifyPdf(pdf)：CMS 验签 / 文档摘要比对 / 证书链 / SES_Signature 专用解析（dataHash 比对 + SM2 验签 + 38540 时间戳字段解析）/ Adobe 逐签名字段验证 / DocTimeStamp 专用解析，输出含 LTV 状态（DSS Certs/OCSPs/CRLs 数量）

X509JsonConverter（101KB，证书 <-> JSON 无损互转）

- certToJson(pemBytes)：X509 全字段 -> JSON（版本/序列号/签名算法 OID+名称/签发者 DN 数组/有效期/使用者/公钥 SPKI+HEX/AKI/SKI/EKU/SAN/AIA/CRLDP/KeyUsage/BasicConstraints/证书策略（CPS+UserNotice）/SCT 列表逐条/签名值/指纹）
- rekeyJsonToPemByIssuer(json, newKp, issuerCert, issuerKey, ...)：密钥更换（按 JSON 重签 TBS，SKI 随新公钥更新）
- renewJsonToPemByIssuer(json, notBefore, notAfter, ...)：续签（变更有效期重签）
- generateKeyPairFromJson(json) / keyTypeFromJson(json)：按 JSON 算法重建密钥对 / 识别密钥类型
- 双向：JSON -> 证书（buildAlgId / buildTbs 等重编码）
- 中文 OID 名称映射（SIG_ALGS、EKU、策略等）

WebAdmin（363KB，内嵌 HTML/CSS/JS + HTTP API）

- main()：独立启动（探测 MAIN 状态）/ port()：SUPORT 端口 / start(mainWebPort)：启动 HttpServer（8 线程执行器 + 写操作 WRITE_LOCK 串行锁）
- 40 个 API 路由（见第七章）
- 前端 15+ 页面：仪表盘（证书链树）、证书库（三选择夹）、签发根证书、签发证书、证书验证、证书操作、调试专区、签名专区（微软代码签名/PDF 文档签名选择夹）、SCT 服务管理（三选择夹）、系统配置、操作日志
-

证书查看器（cert-viewer.js 内嵌）：仿 Windows 证书查看器，三页签，SIG_OID_NAMES 映射（含 ML-DSA-44/65/87）、EKU 中文对照、SCT 列表展示验签、公钥 HEX 显示、自动状态判定（有效期 + 吊销）、颁发者说明按钮

---

# 五、核心运行流程

## 5.1 MAIN 启动流程

- AppPaths.baseDir() 解析 PKI 根 -> Config.load() 读 WEBPORT/OCSP/CRL/CRT/CPS/SUPORT
- Bc.init() 注册 BC Provider
- 启动时自动刷新全部 CRL（RootGetCrl.run(baseDir)）
- 创建 HttpServer（WEBPORT），注册各路由 Handler
- 内嵌 WebAdmin（WebAdmin.start(WEBPORT)，SUPORT 端口）
- 打印启动横幅（全部服务地址 + 自适应说明 + 作者 + 免责声明）

## 5.2 证书签发流程（MakeCert）

```
DN(规范排序) -> 定位颁发者(R:ROOT/C:CA) -> 生成主体密钥(RSA/ECC/SM2/PQC)
-> 构建 TBS(扩展全集)
-> 需要SCT? (非CA 且 EKU 含 1.3.6.1.5.5.7.3.1/2)
    ├── 是: 签发 poison 预证书 -> POST /ct/add -> 取回 SCTList -> 嵌入 SCT 扩展
    └── 否: 直接
-> 按颁发者算法+哈希创建 ContentSigner -> 签名 -> 写证书/私钥 -> 同步 WebPublic/CRT -> 登记 CERTS.txt
```

## 5.3 OCSP 响应流程

```
POST /OCSP -> 解析 OCSPReq(CertID+Nonce)
-> 查 CERTS.txt: 未知序列号 -> 无签名 unauthorized 响应
-> 已知: 通过颁发者序列号 -> OCSP 目录读 序列号_ocsp.crt/.key
-> 摘要自适应: 请求哈希算法(无则SHA1, SM2固定SM3)
-> good / revoked(IMPLICIT RevokedInfo) / unknown -> 签名 -> 响应
```

## 5.4 时间戳流程（自适应根）

```
POST /rfc3161|/timestamp|/timestamp/sha1 -> 解析请求(哈希 OID + messageImprint)
-> 哈希判定: SHA1 -> 随机 rsa_sha1/ecc_sha1; SHA256 -> rsa_sha256/ecc_sha256; SM3 -> sm2_sm3
-> 构建 tstInfo/SignedData -> TSA 证书签名(SM2 摘要切 SM3) -> 响应
```

## 5.5 CT 日志流程

```
POST /ct/add(PEM 预证书+颁发者) -> 解析 -> isSm2Precert 判定
-> pickLogKeys(类型匹配, <=3 组 usable, 随机)
-> 每组: issuer_key_hash=SHA256(颁发者SPKI) + 去poison TBS -> SCT 签名(枚举按类型)
-> SCTList TLS 编码返回; 每密钥一行追加 ctlogs.txt
```

---

# 六、数据文件格式

### 6.1 CERTS.txt（核心登记表）

```
# 序列号  类型(R/C/E)  状态(V/R/N)  原因  时间(2026/09/23-02:26:00)  颁发者ID  算法(RSA/ECC/SM2/PQC)
275bec71dc22a49affc15d65068dfefe E V 0 2026/09/23-02:27:38 f49474498721cfef4e8d456a090ccd23 PQC
```

### 6.2 myCERTS.txt（我的证书，模板 myCERTS.txt.sm）

```
# 序列号    类型        是否存在私钥          算法           生效日期     失效日期
#          R/C/E        Y/N          RSA/ECC/SM2/PQC  日期格式例如 2000/06/17-12:12:12
```

### 6.3 CONFIG.txt

```
WEBPORT=80
OCSP=http://localhost:80/OCSP
CRL=http://localhost:80/CRL/{CID}.crl
CRT=http://localhost:80/CRT/{CID}.crt
CPS=https://gitee.com/wxgshuju/modpkica
CPS_Nonice=http://example.com/nonice
SUPORT=8080
```

说明：{CID} 为序列号占位；CPS_Nonice 支持多行，换行用 \n 存储、生成证书时还原为真实换行。

### 6.4 logs.log

```
<时间> | <毫秒时间戳> | <操作> | <完整命令/提交数据> | <结果/错误详情>

<下一行>
```

只追加，分割符两个换行，Web 操作记录全部提交数据。

### 6.5 CTLOG

- modpkica_ct_list.json：v3 风格 operators[].logs[]（log_id base64、key base64 SPKI、description、type ECDSA/RSA/SM2、state usable/disable、private_key PEM）
- ctlogs.txt：每行 SCT待签名数据取数据摘要||签名时的时间戳||终端证书使用者密钥标识符||终端证书授权密钥标识符||日志公钥的ID

---

## 七、WebAdmin API 路由表

| 路由 | 方法 | 用途 |
| --- | --- | --- |
| /api/status | GET | 服务状态 + 证书统计 |
| /api/certs | GET | CERTS.txt 列表 |
| /api/certs/cacerts | GET | JAVA cacerts 证书列表 |
| /api/logs | GET | 日志读取（按小时筛选） |
| /api/config | GET/POST | 系统配置读写 |
| /api/chain | GET | 证书链（树形图） |
| /api/certjson | GET | 证书 -> JSON |
| /api/certchainjson | GET | 证书链 -> JSON 数组 |
| /api/version | GET | 版本信息 |
| /api/issuers | GET | 颁发者下拉列表 |
| /api/init | POST | 初始化环境 |
| /api/makeroot | POST | 签发根证书 |
| /api/signcert | POST | 签发证书 |
| /api/revoke | POST | 吊销证书 |
| /api/restore | POST | 恢复证书（R->V） |
| /api/debug/ocsp | POST | 模拟 OCSP 请求 |
| /api/debug/tsa | POST | 模拟时间戳请求 |
| /api/rekey | POST | 密钥更换 |
| /api/renew | POST | 续签 |
| /api/crl | POST | 刷新 CRL |
| /api/initocsp | POST | 签发全部 OCSP 证书 |
| /api/inittimestamp | POST | 签发全部 TSA 证书 |
| /api/verify | POST | 证书验证（序列号） |
| /api/my/list | GET | 我的证书列表 |
| /api/my/import | POST | 导入证书（pfx/jks/p12/cer/crt/pem） |
| /api/my/delete | POST | 删除我的证书 |
| /api/verify/file | POST | 本地文件证书验证 |
| /api/signcerts | GET | 签名证书下拉（代码/文档） |
| /api/sign/exe | POST | 微软代码签名 |
| /api/sign/pdf | POST | PDF 文档签名 |
| /api/verify/exe | POST | 代码签名验证 |
| /api/verify/pdf | POST | PDF 签名验证 |
| /api/ctlog/list | GET | CT 密钥列表 |
| /api/ctlog/logs | GET | CT 请求日志 |
| /api/ctlog/gen | POST | 生成 CT 密钥对 |
| /api/ctlog/delete | POST | 删除 CT 密钥 |
| /api/ctlog/enable / /pause | POST | 启用/暂停 |
| /api/ctlog/public | GET | 公网 CT 公钥列表 |
| /api/ctlog/publicFetch | POST | 拉取最新公网数据集 |

## 八、关键标准与算法速查

- RSA 签名 OID：SHA1=1.2.840.113549.1.1.5、SHA256=.11、SHA384=.12、SHA512=.13
- ECDSA 签名 OID：SHA1=1.2.840.10045.4.1、SHA256=1.2.840.10045.4.3.2、P-256=1.2.840.10045.3.1.7
- SM2：签名 1.2.156.10197.1.501（SM3withSM2，含 ZA）、曲线 sm2p256v1=1.2.156.10197.1.301、摘要 SM3=1.2.156.10197.1.401
- ML-DSA（FIPS 204，纯签名）：ML-DSA-44=2.16.840.1.101.3.4.3.17（内置 SHAKE128）、65=.18、87=.19（内置 SHAKE256）；签名长度 2420/3309/4627 字节
- 证书策略 OID：EV SSL=2.23.140.1.1、DV=2.23.140.1.2.1、OV=2.23.140.1.2.2、IV=2.23.140.1.2.3、代码签名普通=2.23.140.1.4.1、代码签名 EV=2.23.140.1.3、TSA=2.23.140.1.4.2、通用文档签名=1.3.6.1.5.5.7.3.36、微软文档签名=1.3.6.1.4.1.311.10.3.12、所有应用策略=1.3.6.1.4.1.311.10.12.1、所有颁发策略=2.5.29.32.0
- SCT（RFC 6962）：OID 1.3.6.1.4.1.11129.2.4.2；precert_entry 待签名 = version|sigType(0)|timestamp|entryType(1)|issuer_key_hash|tbs(去poison/SCT)|extensions
- OCSP（RFC 6960）：revoked 编码 [1] IMPLICIT RevokedInfo；Nonce 回显可选
- DN 排序（X.500/RFC 规范）：C->ST->L->STREET->PostalCode->O->OU->SERIALNUMBER->CN -> EV 字段 -> 自定义 OID

## 九、常用命令速查

```
:: 一键编译 + 自动运行 MAIN
build.cmd 1

:: 初始化环境（目录 + 可选首根）
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.Admin init "CN=Root,O=Org,C=CN" 2048 sha256 RSA

:: 签发根证书（PQC 示例）
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MakeRoot "CN=PQC-Root,O=Org,C=CN" ML-DSA-65 NULL PQC

:: 签发终端证书（PQC 示例，颁发者为上面根）
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MakeCert "CN=www.test.com" ML-DSA-65 NULL 1 3 0 1 "www.test.co

:: 刷新全部 CRL / 签发全部 OCSP / 时间戳证书
java ... modpkica.RootGetCrl
java ... modpkica.Admin signCRL
java ... modpkica.Admin initOCSP RSA
java ... modpkica.Admin initTIMESTAMP SHA256

:: 证书验证
java ... modpkica.CertVerify 1135b5a30.crt
java ... modpkica.Admin verifyCERT 1135b5a30
```

---

# 十、FAQ 速览

- 离线开发：依赖仅 BC185 + lib，无 Maven；改完 build.cmd 即出 out/
- 中文乱码：统一 -Dfile.encoding=UTF-8；脚本注释英文防 cmd GBK 报错
- PQC 证书查看器：签名算法/哈希/公钥均按 ML-DSA 标准显示（OID 2.16.840.1.101.3.4.3.17-19）
- SM2 OCSP 无效签名：v1.1 起按 RFC 6960 IMPLICIT 编码，openssl/BC 均可解析
- SCT 验签：SM2 枚举（7,4）（国密生态）与（7,7）均支持；外部日志公钥缺失显示"未知日志，无法验证"
- 国密 PDF：35275/38540/0031 必须 SM2 证书 + 签章图；38540 与 0031 结构一致（SESv4）
- 重启生效：改源码后需重启 MAIN/WebAdmin 并在浏览器 Ctrl+F5

---

*本分析基于 v1.26（2026-09-26）源码生成，方法与事实均来自实际代码。仅供学习 X.509 ASN.1 编码与 PKI 工程实践使用。*